Intro to Parallel Processes Lab 5 Answers

Name: Thirunavukarasu MuthuValliappan
Student No: 3785346

B2. Performance Testing (Time taken in seconds)

Matrix Size : N	Time Taken for P=1	Time Taken for P=2	Time Taken for P=4
1000	0.043043	0.012490	0.016252
5000	1.059181	0.417697	0.270187
10000	4.443770	2.070087	1.184897
20000	13.979752	9.445834	6.738634

```
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 1 ./lab5 1000
N = 1000, Processes = 1, Time = 0.043043 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 1 ./lab5 5000
N = 5000, Processes = 1, Time = 1.059181 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 1 ./lab5 10000
N = 10000, Processes = 1, Time = 4.443770 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 1 ./lab5 20000
N = 20000, Processes = 1, Time = 13.979752 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 2 ./lab5 1000
N = 1000, Processes = 2, Time = 0.012490 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 2 ./lab5 5000
N = 5000, Processes = 2, Time = 0.417697 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 2 ./lab5 10000
N = 10000, Processes = 2, Time = 2.070087 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 2 ./lab5 20000
N = 20000, Processes = 2, Time = 9.445834 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 4 ./lab5 1000
N = 1000, Processes = 4, Time = 0.016252 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 4 ./lab5 5000
N = 5000, Processes = 4, Time = 0.270187 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 4 ./lab5 10000
N = 10000, Processes = 4, Time = 1.184897 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 4 ./lab5 20000
N = 20000, Processes = 4, Time = 10.771134 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/Lab 5$ mpirun -np 4 ./lab5 20000
N = 20000, Processes = 4, Time = 6.738634 seconds
```

B3. Speedup Calculation

Matrix Size : N	Time Taken for P=1	Time Taken for P=2	Time Taken for P=4
1000	1	3.44	2.64
5000	1	2.53	2.92
10000	1	2.14	3.75
20000	1	1.47	2.07

B4. Questions for Discussion

1. Does the program achieve linear speedup?

No. Even though the data shows a strong speedup of 3.75 for N=10000 with P=4, which is near the ideal value of 4, the efficiency drops significantly to 2.07 as N increases to 20,000. Linear speedup requires the performance to double every time the number of processes doubles, but these results show that as the problem size and process count scale, overhead begins to outpace the computational gains.

2. If not, what factors limit scalability?
Several bottlenecks prevent the program from scaling perfectly. MPI communication overhead is a primary factor, as the time required for MPI_Scatter, MPI_Bcast, and MPI_Gather increases with data size. Additionally, synchronization costs from MPI_Barrier force faster processes to idle while waiting for others. Finally, memory bandwidth limitations occur because the hardware must move large amounts of data, almost 8 times N^2 bytes for a dense matrix between the RAM and the CPU cores.

3. Why does increasing N sometimes improve speedup?
Increasing the matrix size N often improves speedup because it increases the computational intensity of the task. In parallel processing, speedup is a contest between computation (N^2 operations) and communication (linear to N). As N grows, the time spent performing actual calculations on each process increases much faster than the time spent sending messages, allowing the CPU to remain productive for a larger percentage of the total execution time.

4. Why can extremely large values of N cause crashes or segmentation faults?
Extremely large values of N cause crashes, such as Segmentation faults, because of the immense memory footprint of dense matrices. Since each double requires 8 bytes, a matrix where N=20000 requires roughly 3.2 GB of memory. Because Rank 0 is responsible for storing the entire matrix A and the full result vector y, it can easily exceed the physical memory available on a single node, leading the operating system to terminate the program.